FreeDoom: Training a Deep Reinforcement Learning Agent for Deathmatch Combat Environnement

OUALI Yassine, MELLAK Khadija
Department of Intelligent Systems - ENSA Fez

yassine.ouali@usmba.ac.ma, khadija.mellak@usmba.ac.ma

***Abstract*—This paper presents the design, training, and iterative refinement of two deep reinforcement learning agents for first-person combat in a VizDoom deathmatch scenario. The first agent, trained with Proximal Policy Optimization (PPO), operates within a spatially constrained 600×600 unit arena and progressively refines its reward signal across four training experiments, starting from a minimal action set and iteratively adding health tracking, ammunition penalties, and normalized kill rewards. The second agent, Arnold, is trained using a Dueling Double Deep Recurrent Q-Network (Dueling DRQN) on a full deathmatch map with 4 bots across 5 rotating maps, augmented with an auxiliary game-feature perception head. Both approaches are monitored through complementary metrics: episode length, mean episode reward, approximate KL divergence, and PPO clipping fraction for the first agent; TD loss, game-feature loss, mean Q-value, and K/D ratio for the second. After 301 056 timesteps, the PPO agent achieves a 50 % success rate in eliminating both enemies, demonstrating that spatial constraints combined with careful reward shaping enable meaningful policy learning. Arnold, trained over 3.27 million steps, achieves a mean of 16.8 kills per episode with a K/D ratio of 10.68, confirming that value-based recurrent architectures can develop robust combat strategies in more complex settings. Together, these two approaches illustrate the trade-offs between policy gradient and value-based methods in sparse, adversarial environments.**

***Index Terms*—deep reinforcement learning, proximal policy optimization, deep Q-network, VizDoom, reward shaping, first-person shooter, spatial constraint, dueling DRQN**

I. Introduction

Learning to play first-person shooter (FPS) games is a challenging benchmark for deep reinforcement learning (RL). The agent must jointly handle high-dimensional visual observations, sparse rewards, and a stochastic adversarial environment in which multiple enemies act simultaneously. VizDoom [3] provides a controllable API for the Doom engine and has been widely used to study RL agents in combat scenarios.

A recurring challenge in this domain is the curse of dimensionality: a large, open map gives the agent an enormous state space to explore, while the reward signal — based primarily on kills — remains extremely sparse during early training. This paper investigates two complementary strategies to address this challenge.

The first strategy, based on **Proximal Policy Optimization (PPO)** [1], introduces a *spatial constraint* of 300 units of permitted zone measured as a Manhattan distance from the agent's spawn point. The agent is penalized for exceeding this boundary, effectively creating a bounded arena that forces engagement with enemies rather than aimless wandering. We describe four successive training experiments, each motivated by limitations identified in the previous one, progressing from a minimal five-action baseline to a fully shaped reward function with kill bonuses, damage penalties, ammo costs, and survival incentives. Training metrics are monitored through TensorBoard [4]. The final PPO configuration, `doom_ppo_v1_3`, achieves a 50 % success rate over 10 test episodes with a mean reward of 0.623 and an average episode length of 126.5 steps after 301 056 training timesteps.

The second strategy, based on a **Dueling Double Deep Recurrent Q-Network (Dueling DRQN)** [6], [7], removes the spatial constraint and trains on a full deathmatch map against 4 bots across 5 rotating maps. This agent, named Arnold, augments the standard DQN architecture [8] with a shared LSTM memory layer and an auxiliary game-feature prediction head that forces the visual encoder to learn perceptually meaningful representations. Arnold is trained for 3.27 million environment steps and achieves a mean of 16.8 kills per episode, a K/D ratio of 10.68, and a final greedy-evaluation K/D of 5.40, demonstrating that recurrent value-based architectures scale effectively to more complex combat settings.

Taken together, the two approaches highlight a fundamental trade-off in deep RL for FPS games: PPO offers faster iteration and interpretable convergence diagnostics but requires careful environment simplification to learn meaningful behavior; Dueling DRQN scales to richer environments at the cost of greater architectural complexity and a significantly longer training budget.

The remainder of this paper is organized as follows. Section II presents the theoretical background of PPO and Dueling DRQN, the two algorithms used in this work. Section III reviews related work and discusses the contributions of this paper relative to prior approaches. Section IV describes the PPO-based approach in full: environment design, reward function, training experiments, and evaluation results. Section V presents the Arnold agent, its architecture, training, and evaluation. Section VI discusses limitations, and Section VII concludes.

II. Background

This section reviews the two core deep reinforcement learning algorithms employed in this work: Proximal Policy Optimization for the PPO agent, and the Dueling Double Deep Recurrent Q-Network for the Arnold agent.

A. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) [1] is an on-policy actor-critic algorithm that limits the size of each policy update through a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big], \tag{1}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimate. The clipping parameter ε (typically 0.2) prevents excessively large updates that could destabilize training. PPO combines the sample efficiency of trust-region methods with the simplicity of first-order gradient optimization, making it well-suited for environments where data collection is costly and policy stability is paramount.

B. Dueling Double Deep Recurrent Q-Network (Dueling DRQN)

The Arnold agent is built on three architectural extensions of the foundational DQN algorithm [8].

Dueling Networks [6] decompose the Q-value estimate into a state-value stream $V(s)$ and an advantage stream $A(s, a)$:

$$Q(s, a; \theta) = V(s; \theta_V) + \left(A(s, a; \theta_A) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s, a'; \theta_A)\right) \tag{2}$$

This decomposition improves value estimates in states where the choice of action has little impact, which is common during navigation between combat encounters.

Deep Recurrent Q-Networks (DRQN) [7] replace the fully connected layers of DQN with a Long Short-Term Memory (LSTM) layer, enabling the agent to maintain a memory of recent events — such as enemy positions and shot trajectories — beyond the capacity of a fixed-size frame stack alone. This is particularly important in partially observable environments like VizDoom deathmatch, where enemies can be temporarily out of view.

Double DQN mitigates the Q-value overestimation bias of standard DQN by decoupling action selection from action evaluation: the online network selects the greedy action while the frozen target network evaluates it. Together these three components form the Dueling DRQN architecture used by Arnold.

III. Related Work

A. Playing FPS Games with Deep Reinforcement Learning

The most directly relevant prior work is that of Lample and Chaplot [2], who trained agents to play VizDoom deathmatch using a Deep Recurrent Q-Network augmented with an auxiliary game-feature prediction head. Their key insight was that forcing the visual encoder to simultaneously predict binary game variables — such as whether an enemy is currently visible — acts as a perceptual regularizer, yielding richer and more semantically grounded feature representations than Q-value optimization alone. Their agent was evaluated on multi-player deathmatch maps and demonstrated competitive performance against built-in bots, establishing a strong baseline for the VizDoom deathmatch setting.

B. Contributions of This Work

Our paper builds on the framework of Lample and Chaplot [2] in several ways while introducing a complementary policy-gradient perspective.

a) Spatial constraint and reward shaping for PPO.: Rather than relying solely on a value-based recurrent architecture, we investigate whether a simpler on-policy method (PPO) can learn meaningful combat behavior when the exploration space is bounded by a 600×600 unit spatial constraint. This constraint has not been explored in the prior VizDoom literature and offers a principled mechanism to increase encounter density during early training.

b) Iterative reward engineering.: We document four successive training experiments, each correcting a specific reward-engineering error identified in the previous run — including a misattributed damage signal and an oversized kill reward — providing a reproducible account of the reward shaping process that is rarely reported in detail in prior work.

c) Dual-method comparison.: By training both a PPO agent and a Dueling DRQN agent (Arnold) on the same VizDoom platform under different environment configurations, we provide a direct empirical comparison of policy-gradient and value-based approaches in the FPS deathmatch domain. Arnold's architecture directly replicates the game-feature head proposed by Lample and Chaplot [2], allowing us to validate their perceptual regularization hypothesis in a reproducible setting and to contrast it with the spatial-constraint strategy used for the PPO agent.

IV. PPO-Based Approach

This section describes in full the environment design, reward function, iterative training experiments, and evaluation results of the PPO-based agent.

A. Environment Design

1) VizDoom Configuration: The environment is built on top of a custom VizDoom deathmatch scenario defined by the configuration file `deathmatch_mine.cfg` and the associated WAD file `deathmatch_mine.wad`. The game difficulty is set to skill level 3 (intermediate), and each episode is capped at **4 200 game tics** (episode_timeout), providing a hard time limit that prevents indefinitely long episodes. The screen resolution is set to 320×240 during training for computational efficiency; the HUD is disabled to avoid introducing non-essential visual features into the observation that will disturb the agent learning, like the HP bar, ammo count and the player avatar. A separate visible window at 640×480 is available for manual inspection only and is disabled during automated training runs.

Two execution modes are supported:

- `ASYNC_PLAYER` — used during training; the game advances asynchronously and does not wait for the agent, which speeds up data collection.
- `PLAYER` — used during visual inspection, especially during test. The game then runs synchronously at full resolution.

The full environment is implemented as a `gymnasium.Env` wrapper class (`DoomEnv`), making it directly compatible with Stable-Baselines3 and any other Gymnasium-compliant RL framework [5]. The wrapper encapsulates episode management, observation pre-processing, action dispatch, reward computation, and termination logic, providing a clean interface for training the PPO algorithm.

2) Game Variables: Eight game variables are declared in the configuration file and exposed to the reward function at each step. Table I lists each variable together with its index in the `state.game_variables` array, its type, and its role in the reward shaping pipeline.

TABLE I
GAME VARIABLES EXPOSED BY THE VIZDOOM CONFIGURATION

Idx	Variable	Type	Role
0	`KILLCOUNT`	Int	Kill reward signal
1	`HEALTH`	Int	Damage & low-health penalty
2	`ARMOR`	Int	Reserved, unused
3	`SELECTED_WEAPON`	Int	Reserved, unused
4	`S_WEAPON_AMMO`	Int	Ammunition penalty
5	`POSITION_X`	Float	Spatial constraint
6	`POSITION_Y`	Float	Spatial constraint
7	`DAMAGECOUNT`	Int	Damage-received penalty

An important subtlety is that `DAMAGECOUNT` tracks *cumulative damage received by the agent* since the start of the episode, not damage inflicted on enemies. The reward function therefore uses the *delta* $\Delta d_t = \texttt{DAMAGECOUNT}_t - \texttt{DAMAGECOUNT}_{t-1}$ to obtain per-step incoming damage. Tracking outgoing damage would require custom ACS scripting inside the WAD scenario and was outside the scope of this work.

3) Observation Space: The raw screen buffer returned by VizDoom has shape (C, H, W) with $C \in \{1, 3\}$ depending on the color mode. The `preprocess()` method of `DoomEnv` applies the following pipeline at every step:

1) **Transpose** — if the buffer is in (C, H, W) format it is transposed to (H, W, C).
2) **Grayscale conversion** — if a single-channel buffer is detected it is converted to RGB using `cv2.COLOR_GRAY2RGB`.
3) **Channel clipping** — if more than three channels are present, only the first three are retained.
4) **Resize** — the frame is down-sampled to 84×84 pixels using OpenCV's default interpolation, following the pre-processing standard established by Lample and Chaplot [2].
5) **dtype cast** — the output is cast to `uint8` to match the Gymnasium observation space declaration.

The final observation space is declared as:

$$\mathcal{O} = \{0, 1, \ldots, 255\}^{84 \times 84 \times 3} \tag{3}$$

and registered with Gymnasium as a `spaces.Box(low=0, high=255, shape=(84,84,3), dtype=np.uint8)`. When an episode terminates (agent death or zone violation), a black frame of zeros is returned as a terminal observation.

4) Action Space: The available buttons declared in the configuration file are mapped to a **discrete action space** of eight macro-actions, as listed in Table II. Each macro-action is represented internally as a binary vector of length n (the total number of configured buttons), where entry i is 1 if button i is pressed and 0 otherwise.

TABLE II
DISCRETE ACTION SPACE — EIGHT MACRO-ACTIONS

Index	Action	Buttons active
0	Move forward	`MOVE_FORWARD`
1	Move backward	`MOVE_BACKWARD`
2	Strafe left	`MOVE_LEFT`
3	Strafe right	`MOVE_RIGHT`
4	Turn left	`TURN_LEFT`
5	Turn right	`TURN_RIGHT`
6	Attack	`ATTACK`
7	Move forward + Attack	`MOVE_FORWARD` + `ATTACK`

The action space is registered with Gymnasium as `spaces.Discrete(8)`. Each selected action is executed for **4 consecutive game tics** (frame skip $= 4$) via `game.make_action(action, 4)`. Frame skipping reduces the effective decision frequency, decreases temporal correlation between consecutive observations, and significantly accelerates wall-clock training time.

5) Spatial Constraint: To bound the exploration space and prevent the agent from wandering across the entire map — which would make enemy encounters extremely sparse — we introduce a **spatial termination constraint**. At the beginning of each episode the agent's starting coordinates (x_0, y_0) are recorded from `POSITION_X` and `POSITION_Y`. At every subsequent step, if the axis-aligned distance from the starting position exceeds 300 game units in either dimension:

$$|x_t - x_0| > 300 \quad \text{or} \quad |y_t - y_0| > 300, \tag{4}$$

the episode is terminated immediately and a zone penalty of $r_{\text{zone}} = -1.0$ is applied. This constraint effectively defines a 600×600 unit bounding box centered on the spawn point, creating a pseudo-arena that keeps the agent near the enemies while still allowing meaningful navigation.

B. Reward Function

The reward function is the central mechanism through which the agent learns its combat strategy. Its design must balance three competing objectives: encourage the agent to **actively engage and eliminate enemies**, penalize **passive or evasive**

behavior (standing still, fleeing indefinitely), and discourage **reckless play** (absorbing damage, wasting ammunition, leaving the combat zone). Each component was assigned a value chosen to reflect this priority ordering, so the kill bonus dominates all other signals, while the auxiliary penalties are kept small enough to shape behavior without overshadowing the primary objective.

The total per-step reward is:

$$r_t = r_{\text{kill}} + r_{\text{surv}} + r_{\text{time}} + r_{\text{dmg}} + r_{\text{ammo}} + r_{\text{zone}} + r_{\text{death}} \quad (5)$$

Table III details each component, its trigger condition, its value, and the behavioral objective it encodes.

TABLE III
REWARD FUNCTION COMPONENTS AND THEIR DESIGN RATIONALE

Component	Condition	Value	Objective
Kill bonus	Per new kill	$+1.0$	Primary goal; dominates all other terms
Survival bonus	Every step	$+0.005$	Discourages passive death; rewards staying alive
Time penalty	Every step	-0.001	Discourages idling or fleeing; promotes efficiency
Damage penalty	Per damage unit Δd	$-0.02\times\Delta d$	Penalizes absorbing hits; promotes evasion
Ammo penalty	Per ammo unit Δa	$-0.003 \times \Delta a$	Discourages random firing; promotes accurate shots
Zone penalty	Exit 300-unit bound	-1.0	Prevents indefinite retreat from enemies
Death penalty	Agent dies	-0.3	Penalizes terminal failure
Low-health penalty	Health < 25	-0.01/step	Urges the agent to avoid dangerous situations

Design Rationale: The kill bonus is set to $+1.0$ per kill, a value chosen after observing that a larger reward of $+50$ used in earlier experiments caused the kill signal to completely dominate the auxiliary penalties, leading the agent to ignore health, ammo, and spatial constraints entirely. Normalizing to $+1.0$ ensures that the agent can still accumulate a meaningful positive return from kills while remaining sensitive to the shaping signals.

The survival bonus ($+0.005$) and time penalty (-0.001) act as opposing forces; together they yield a net positive reward of $+0.004$ per step for an agent that stays alive without acting, which is intentionally small so that passive survival is rewarded less than a single kill. Their combination discourages two failure modes observed during early training: the agent dying immediately by engaging recklessly, and fleeing enemies indefinitely to accumulate survival bonuses.

The damage and ammo penalties are deliberately scaled an order of magnitude below the kill bonus. A damage penalty of -0.02 per unit means that absorbing a typical hit of 10–20 damage points costs between -0.2 and -0.4, roughly one third to one half of a kill reward — enough to incentivize evasion without making the agent too passive. Similarly, the ammo penalty of -0.003 per round makes firing ten consecutive missed shots equivalent to a penalty of -0.03, nudging the agent toward accurate fire without preventing it from shooting at all.

The zone penalty (-1.0) is the largest single-step negative signal and terminates the episode immediately, making boundary violation a hard constraint rather than a soft discouragement. This directly addresses the failure mode of an agent that learns to run away from enemies to avoid taking damage. However, it was kept moderate to avoid conditioning the agent too strongly to a confined space, which could hinder generalization when the spatial constraint is relaxed or removed in future experiments on larger maps.

C. Training Experiments

We conducted four successive experiments, each addressing weaknesses identified in the previous run. All experiments use Stable-Baselines3's PPO implementation with a CNN policy.

1) Essai 1 — Baseline: The first experiment used only five actions (attack, move forward/backward, turn left/right) and four game variables (KILLCOUNT, POSITION_X, POSITION_Y, DAMAGECOUNT). No health or ammo tracking was present. This run served as a proof-of-concept to verify that the environment loop, reward calculation, and episode termination worked correctly.

2) Essai 2 — Damage Clarification (doom_ppo_v1_1): A key misunderstanding was identified: the `DAMAGECOUNT` variable tracks *damage received by the agent*, not damage inflicted on enemies. Using it as a bonus would penalize the agent for being hit rather than rewarding aggressive behavior. Essai 2 corrected this by treating the variable strictly as a penalty signal. Training ran to approximately 100 000 steps. The mean episode length oscillated between 70 and 85 steps, showing early but unstable learning.

3) Essai 3 — Extended Training (doom_ppo_v1_2): Building on the corrected reward, Essai 3 introduced a kill reward of $+50$ (later identified as too large), a survival bonus, and a time-spending penalty to encourage efficiency. Training ran for **1 000 000 timesteps**, approximately 38 hours.

To track the agent's performance through time, we used two essential metrics. `ep_len_mean` denotes the mean number of steps per episode averaged over all episodes completed within a given rollout window; a higher value indicates that the agent survives longer before dying, leaving the combat zone, or reaching the episode timeout. `ep_rew_mean` describes the mean cumulative reward accumulated per episode over the same rollout window; a positive and increasing trend indicates that the agent is learning to collect more reward than it incurs in penalties, reflecting genuine behavioral improvement.

The mean reward `ep_rew_mean` reached peaks of 60–66 at around 450k–480k steps before degrading. The approximate KL divergence grew from ≈ 0.01 at the start to spikes of 0.15–0.23 near the end, indicating increasingly aggressive policy updates not fully contained by clipping. The clipping

TABLE IV
ESSAI 3 — EPISODE_LENGTH_MEAN TRAINING PHASES

Phase	Steps	Observation
Initialization	2k–30k	Random behavior, quick deaths
First improvement	30k–70k	Longer survival
Strong progression	70k–200k	ep_len reaches 100–135
High plateau	200k–550k	Stable behavior
Degradation	550k–620k	Sudden drop (125→65)
Partial recovery	620k–700k	Rebounds to 65–115
Final stabilization	700k–1M	Stabilizes at 90–105

fraction rose from ≈ 0.13 to values often above 0.35, confirming that PPO was regularly operating in its constraint regime.

4) Essai 4 — Normalized Reward Shaping (doom_ppo_v1_3): Essai 4 introduced the following refinements:

- Kill reward normalized to $+1.0$ per kill (from $+50$), preventing kill dominance over auxiliary signals.
- Ammo penalty (-0.003 per round spent) to discourage random firing.
- Health variable added to derive a damage penalty (-0.02 per damage unit received).
- Correct kill counter display (actual kill count, not cumulative reward increments of 50/100).
- Correct episode reward logging.

Training ran for **300 056 timesteps** ($\approx$13.85 hours), three times faster than Essai 3 due to the reduced budget.

D. Results and Evaluation

1) Training Curves — Essai 4: Figure 1 shows `ep_len_mean` over 300k steps. The curve rises from an initial average of ≈ 70 steps per episode to a final smoothed value of $\approx$ 103.6 steps, representing a $\approx$ 48 % increase in survival duration. Strong oscillations persist throughout, indicating that the policy has not fully converged within this budget.

Figure 2 shows `ep_rew_mean`. The reward begins at approximately -0.2 (penalties dominating), reaches a peak of ≈ 0.45 around 150k steps, then stabilizes near 0.35–0.40 at the end. The final positive mean reward confirms net beneficial behavior relative to random play.

2) PPO Diagnostic Metrics: Two internal PPO metrics are monitored to assess the health of the training process beyond the rollout rewards.

Approximate KL divergence (`train/approx_kl`) measures the statistical distance between the old policy $\pi_{\theta_{\text{old}}}$ and the updated policy π_θ after each gradient step. A low KL indicates conservative updates; a high KL indicates that the policy is changing aggressively. In well-tuned PPO runs this value typically stays below 0.02.

Clipping fraction (`train/clip_fraction`) measures the proportion of sampled transitions whose probability ratio

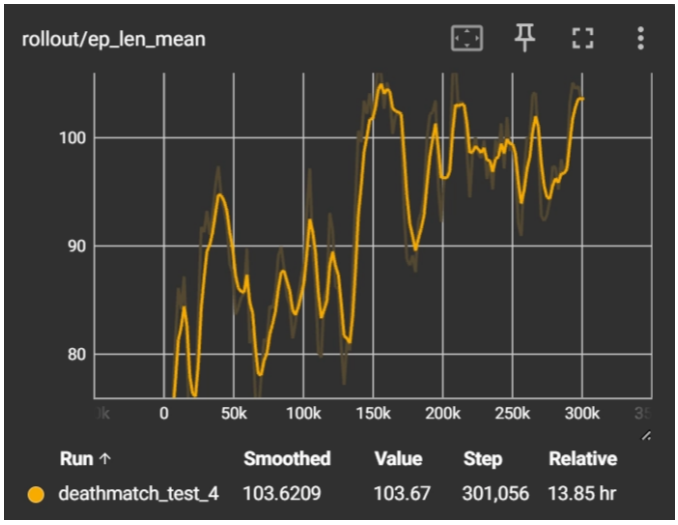

Fig. 1. Mean episode length during Essai 4 training. Despite oscillations, the trend is upward, reflecting that the agent learns to survive longer over time.

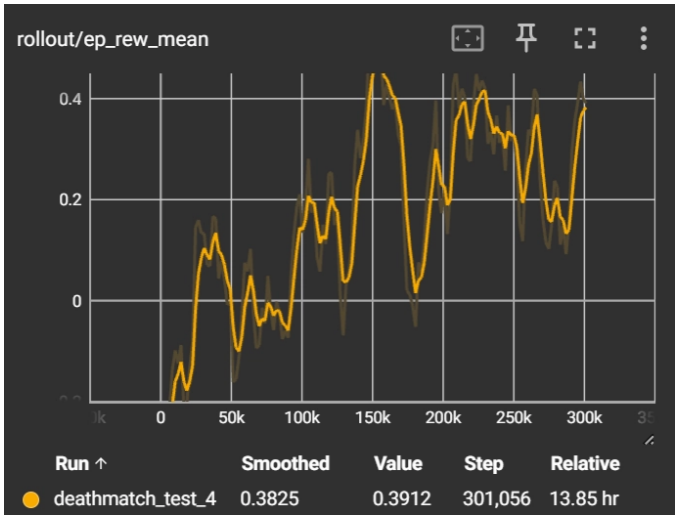

Fig. 2. Mean episode reward during Essai 4 training. The upward trend from negative to positive values shows that the agent progressively learns to accumulate more reward than it incurs in penalties.

$r_t(\theta)$ fell outside the clipping range $[1-\varepsilon, 1+\varepsilon]$ and was therefore truncated. It is defined as:

$$f_{\text{clip}} = \frac{\#\text{ clipped samples}}{\#\text{ total samples}} \tag{6}$$

A healthy PPO run maintains f_{clip} around 0.02–0.10, and values persistently above 0.30 indicate that the policy is attempting updates larger than the clipping mechanism can comfortably constrain.

Figure 3 shows the approximate KL divergence over 300k training steps. Starting at ≈ 0.009 at 4k steps, the KL rises progressively — a normal sign that the policy is diverging from its initialization and learning new strategies. It reaches 0.10–0.18 around 80k–120k steps, coinciding with the period of strongest reward growth, then continues to climb to values of 0.20–0.40 between 190k and 300k steps. Despite these high

values, training does not diverge: `ep_rew_mean` remains positive throughout, confirming that the clipping mechanism successfully contains the updates.

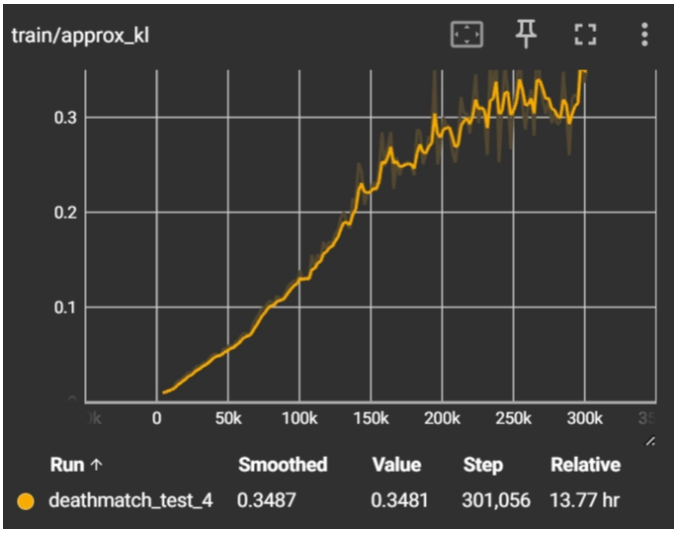

Fig. 3. Approximate KL divergence during Essai 4. The progressive increase reflects active policy learning; late-stage spikes up to 0.40 indicate overly aggressive updates not fully contained by clipping.

Figure 4 shows the clipping fraction over the same training run. It rises sharply from 0 to ≈ 0.50 within the first 70k steps, peaks at ≈ 0.52 between 70k and 120k steps — the same window in which `ep_rew_mean` and `ep_len_mean` grow fastest — then gradually settles to a plateau of 0.43–0.48 for the remainder of training. The fact that f_{clip} never falls below 0.40, far above the healthy range of 0.02–0.10, confirms that the policy is still undergoing significant change at 300k steps and has not yet converged. Table V summarizes the key observations.

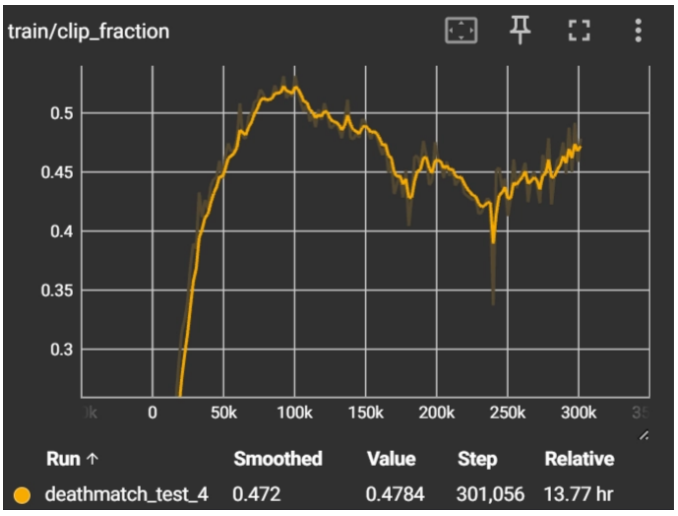

Fig. 4. Clipping fraction during Essai 4. The peak at ≈ 0.52 coincides with the fastest learning phase; the late-stage plateau at 0.43–0.48 indicates that convergence has not been reached within 300k steps.

TABLE V
CLIPPING FRACTION — KEY OBSERVATIONS

Observation	**Interpretation**
Peak at ≈ 0.52	Period of strongest policy learning
Plateau at 0.45–0.48	Policy still evolving; no convergence
No drop toward 0	Training budget insufficient
No explosion >0.6	Training globally stable

3) Test Evaluation (10 Episodes): Table VI reports per-episode results from the final `doom_ppo_v1_3` model.

TABLE VI
TEST RESULTS — DOOM_PPO_V1_3 (10 EPISODES)

Episode	Reward	Kills	Steps	Outcome
1	2.180	2	167	Success
2	-0.324	0	84	Failure
3	2.180	2	167	Success
4	-0.617	0	107	Failure
5	-0.818	0	50	Failure
6	2.145	2	154	Success
7	-0.004	0	168	Failure
8	2.180	2	167	Success
9	-0.873	0	34	Failure
10	2.180	2	167	Success

Key statistics: 5/10 episodes succeeded (both enemies eliminated), mean reward $= 0.623$, mean episode length $= 126.5$ steps, max kills per episode $= 2$. Successful episodes cluster tightly at reward ≈ 2.18 and 167 steps, suggesting the agent has internalized a consistent winning strategy. Failed episodes divide into three main causes: agent death (3 episodes), zone boundary violation (3 episodes), and inability to engage enemies (5 episodes with 0 kills).

V. DQN-BASED APPROACH: ARNOLD AGENT

While the PPO approach described in Section IV focuses on policy gradient optimization within a spatially constrained single-zone arena, a complementary agent **Arnold** was trained using a value-based method, which is the Dueling Double Deep Recurrent Q-Network (Dueling DRQN) [6], [7]. Arnold operates on a full deathmatch map with 4 bots across 5 rotating maps, receiving denser rewards and an auxiliary perception head, making it a structurally richer baseline for comparison.

A. Environment and Configuration

Arnold is trained on VizDoom's deathmatch scenario with 4 bots rotating across 5 maps. Table VII summarizes the key hyperparameters and architectural choices.

B. Reward Shaping

Arnold's reward function combines a dominant kill signal with several auxiliary components designed to encourage active movement and discourage passive camping as follows :

$$r_t = r_{\text{kill}} + r_{\text{death}} + r_{\text{dist}} + r_{\text{stale}} + r_{\text{health}} \tag{7}$$

TABLE VII
ARNOLD — TRAINING CONFIGURATION

Parameter	Value
Algorithm	Dueling DRQN (Double DQN + LSTM)
Environment	VizDoom deathmatch, 4 bots, Maps 1–5
Observation	108×60 RGB, 4-frame stack, frame skip 4
Action space	Factored: turn+move, attack, strafe
Replay buffer	100 000 transitions
Batch / horizon	32 sequences, 4 recurrent unrolls
Learning rate	2×10^{-4} (Adam)
Target update	Every 1 250 train steps ($\approx$5 000 env steps)
ε schedule	$1.0 \rightarrow 0.10$ over 500 000 env steps
Discount γ	0.99
Total steps	3 271 025

TABLE VIII
ARNOLD — REWARD FUNCTION COMPONENTS

Component	Value	Objective
Kill bonus	$+30$	Primary combat objective
Death penalty	-10	Penalize terminal failures
Distance bonus	$+0.005 \times d$	Encourage map exploration
Stale penalty	-0.1/step	Discourage standing still
Health pickup	$+1$	Reward self-preservation

The kill bonus is set to $+30$, significantly higher than the $+1$ used in the PPO approach, reflecting the denser and longer episodes in the full deathmatch setting. The distance bonus and stale penalty act as complementary anti-camping mechanisms. Together they push the agent to continuously explore the map rather than hold a fixed position. A Pearson correlation of $r = 0.984$ between episode reward and kill count confirms that the shaping is well-aligned with the primary objective.

C. Architecture: Dueling DRQN with Game Features

Arnold extends the standard DQN architecture in two ways. First, it adopts the **Dueling network** architecture [6], which separates the Q-value estimate into a state-value stream $V(s)$ and an advantage stream $A(s, a)$ as described in Section II. This decomposition improves value estimates in states where the choice of action has little impact, which is common during navigation between combat encounters.

Second, a shared **LSTM** layer [7] processes the sequence of frame-stacked observations, enabling the agent to maintain a memory of recent events (enemy positions, shot trajectories) beyond what the $108 \times 60 \times 4$ frame stack alone can encode.

Third, an **auxiliary game-feature head** branches off the shared LSTM hidden state to predict binary visibility indicators — whether a target or enemy is currently visible. This head, directly inspired by Lample and Chaplot [2], acts as a perceptual regularizer, forcing the visual encoder to extract semantically meaningful features beyond pure Q-value optimization.

D. Training Results

Learning Curves: Two primary rollout metrics are monitored throughout training: episode reward (`ep_reward`) and kill count per episode (`ep_kills`), together providing complementary views of combat progress.

Figure 5 shows the cumulative episode reward over 3.27M training steps. The reward rises steeply during the first 500k steps as the agent transitions from pure exploration ($\varepsilon = 1.0$) toward exploitation, then stabilizes around a mean of 503.0 with a peak of 1 307.3. The persistently high variance is expected in deathmatch, because episodes vary in duration (up to 2 100 game ticks) and opponent behavior is non-stationary.

Fig. 5. Cumulative episode reward during Arnold's training. The upward trend during the first 500k steps reflects the transition from exploration to exploitation; the high variance is inherent to the deathmatch setting.

Figure 6 shows the kill count per episode. Starting at 1–5 kills during the exploration phase, the smoothed curve rises to a mean of 16.8 kills per episode by the end of training, with a single-episode peak of 43 kills against 4 bots, demonstrating that Arnold can decisively dominate the arena under favorable conditions.

Fig. 6. Kill count per episode during Arnold's training. The monotonically increasing smoothed trend confirms genuine improvement in combat ability across all four training phases.

Kill/Death Ratio: Arnold maintains a mean K/D ratio of 10.68 over all training episodes, with 86.8 % of episodes ending with K/D > 1. Under the greedy evaluation policy ($\varepsilon = 0$), the final checkpoint achieves a K/D of 5.40 over 10 episodes across all 5 maps, confirming that the learned policy generalizes beyond the training distribution and is not an artifact of stochastic exploration.

E. Loss Dynamics

Two loss functions are jointly optimized during training.

The **TD loss** (SmoothL1/Huber) measures the error between the online Q-network's predictions and the Bellman bootstrap targets from the frozen target network. After an initial high-loss phase, it stabilizes in the range $[0.072, 0.346]$ with a mean of 0.2384. A non-zero plateau is expected and healthy; the target network is refreshed every 1 250 training steps, preventing convergence to zero.

The **game-feature loss** (GF loss) measures the prediction error of the auxiliary visibility head. It begins at ≈ 0.45 and decays to a plateau near 0.255, confirming that the shared LSTM encoder learns to represent perceptually meaningful scene content. The co-convergence of GF and TD losses after ≈ 1M steps is a positive sign that the two optimization objectives do not interfere. Figure 7 shows a comparative view of the two loss curves.

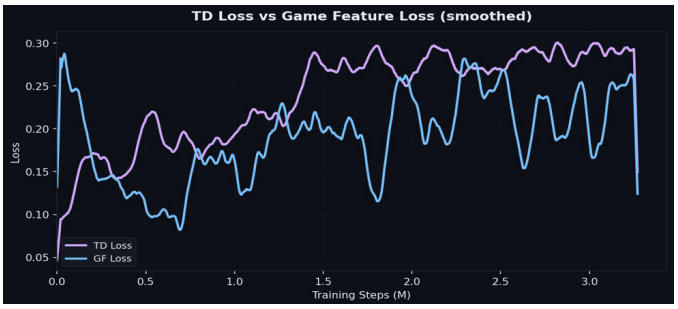

Fig. 7. Overlaid TD and GF losses (smoothed). The relative magnitudes and convergence trajectories are compared directly.

F. Kill and Reward Distribution Analysis

To characterize the behavioral consistency of Arnold beyond aggregate means and peak values, we analyze the statistical distributions of kill count and episode reward over a representative sample of 2 000 training episodes. This distributional perspective complements the time-series view of Section V by revealing the shape of the agent's performance envelope, whether gains are concentrated in rare high-scoring episodes or spread uniformly across the population.

Kill Count Distribution: Figure 8 shows the empirical histogram of kills per episode over the 2 000-episode sample, overlaid with a kernel density estimate. The distribution is approximately unimodal with a right skew, centered near the sample mean of 16.8 kills and a mode in the 14–18 kill bin. The right tail, extending to a maximum of 43 kills, reflects occasional dominant episodes in which Arnold achieves near-total map control against the four bots; such outliers constitute fewer than 5 % of episodes and do not inflate the mean substantially. The absence of a significant mass near zero confirms that the learned policy is robust, and Arnold reliably achieves a positive kill count regardless of map configuration or bot spawn positions. The moderate standard deviation ($\sigma \approx 6.1$ kills) reflects the inherent stochasticity of the deathmatch environment rather than policy instability, as evidenced by the stable phase-by-phase statistics reported in Table IX.

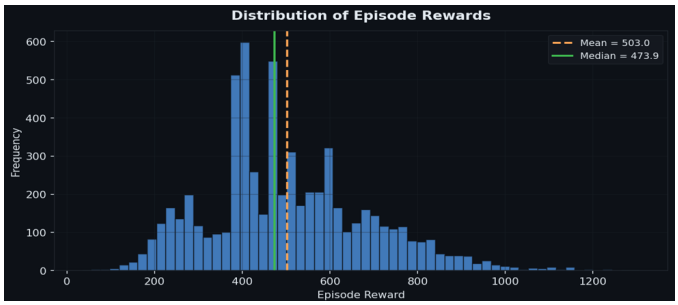

Fig. 8. Distribution of kills per episode over a 2 000-episode sample. The right-skewed unimodal shape, centered near 16.8 kills, confirms consistent combat performance with rare high-scoring outliers extending the right tail.

Kill–Reward Joint Distribution: Figure 9 visualizes the joint relationship between kill count and cumulative episode reward over the same 2 000-episode sample. Individual episodes are rendered as a scatter plot; a linear regression line is overlaid to summarize the global trend. The near-linear alignment of the cloud (Pearson $r = 0.984$) confirms that episode reward is almost entirely determined by kill count, validating the reward design: auxiliary components (distance bonus, stale penalty, health pickup) collectively contribute a negligible fraction of the total return compared to the dominant kill bonus of $+30$ per kill.

Two structural features of the scatter are noteworthy. First, the intercept of the regression line is slightly negative, indicating that in the rare episodes where Arnold scores zero or one kill, the death penalties and stale penalties push the net reward below zero. Second, the variance of the reward conditional on a fixed kill count widens slightly at higher kill values, reflecting differences in episode duration and survival across maps: a 17-kill episode on a compact map with short respawn paths yields a different auxiliary-component total than the same kill count on a larger map with extended navigation phases.

Fig. 9. Kills versus cumulative episode reward over 2 000 episodes. The near-linear relationship (Pearson $r = 0.984$) confirms that the reward signal is dominated by the kill bonus, with auxiliary components introducing only minor variance around the regression line.

Taken together, the two distributions confirm that Arnold's policy is both consistent — high kill counts are the norm,

not the exception — and well-calibrated to its reward signal, providing a sound basis for interpreting the aggregate metrics reported in Table X.

G. Value Function and Exploration

The mean Q-value grows monotonically from 0.311 at initialization to a peak of 24.07, before settling at 20.07 at the end of training — a $65\times$ increase. The smooth trajectory, free of sharp spikes, indicates that the target network update frequency (every 1 250 steps) successfully dampens the Q-value overestimation spirals that are a known failure mode of vanilla DQN.

The ε-greedy exploration schedule decays linearly from $\varepsilon = 1.0$ to the floor of $\varepsilon = 0.10$ over the first 500 000 environment steps, then holds the floor for the remaining 2.77M steps. The 10 % residual exploration is crucial, it prevents the policy from over-specializing to locally optimal behaviors (e.g., camping a single corridor) discovered early in training, and limits covariate shift in the replay buffer.

H. Phase-by-Phase Analysis

Table IX divides the 3.27M-step training run into four equal quartiles and reports the mean and standard deviation of kills per episode in each phase.

TABLE IX
ARNOLD — KILL STATISTICS BY TRAINING PHASE

Phase	Steps	Mean kills	Std	Max
Phase 1	0 – 0.82M	16.81	6.55	43
Phase 2	0.82 – 1.63M	17.32	6.44	43
Phase 3	1.63 – 2.45M	16.07	5.50	38
Phase 4	2.45 – 3.27M	17.11	5.84	42

Mean kills remain stable across all four phases ($\approx$16–17), while the standard deviation narrows from Phase 1 to Phase 4, confirming that the policy stabilizes over time without catastrophic forgetting. Key milestones are identified at $\approx$500k steps (end of ε-annealing), $\approx$1M steps (GF loss convergence), and $\approx$2M steps (Q-value plateau and variance reduction).

I. Evaluation Performance

Every 20 000 training steps, Arnold is evaluated for 300 game ticks over 10 greedy episodes ($\varepsilon = 0$). Over 163 evaluation checkpoints, the mean evaluation kill count is 5.00 per episode, peaking at 6.6 kills at the 240k-step checkpoint and finishing at 5.4 kills at the final checkpoint. The evaluation K/D ratio mirrors this trend, with a final value of 5.40. The lower evaluation kills relative to training (5.4 vs. 16.8) reflect the shorter episode duration (300 ticks vs. up to 2 100 ticks in training) rather than any policy degradation.

Table X summarizes the final performance statistics.

TABLE X
ARNOLD — FINAL PERFORMANCE SUMMARY

Metric	Value	Context
Total training steps	3.27M	6 215 episodes
Mean episode kills	16.83	Training
Peak episode kills	43	Training
Mean K/D ratio	10.68	Training
Eval kills (final)	5.4	Greedy, 300 ticks
Eval K/D (final)	5.40	Greedy policy
Mean episode reward	503.0	Training
Peak episode reward	1 307.3	Training
Mean TD loss	0.2384	Stable plateau
Mean GF loss	0.1915	Auxiliary head
Peak mean Q	24.07	Training
Final mean Q	20.07	Stable
Reward–kill correlation	0.984	Pearson r

VI. LIMITATIONS

Several limitations of the current work merit discussion.

a) No outgoing damage signal.: VizDoom's default deathmatch configuration does not expose the damage the agent inflicts on enemies, and incorporating this signal would require custom ACS scripting in the Doom engine's scenario files, which was outside the scope of this work. As a result, both agents can only infer combat effectiveness through kill events, providing a very sparse guidance signal during enemy encounters. Future work should instrument the WAD scenario with a custom damage sensor to provide a richer intermediate reward between shots fired and kills registered.

b) Spatial constraint as an approximation.: The 300-unit Manhattan distance bound used by the PPO agent is a rough proxy for keeping the agent near the action. A more principled approach would use curriculum learning to gradually expand the accessible area as the agent's skill improves, or define the zone dynamically around detected enemies. The hard zone boundary also risks conditioning the agent too strongly to a confined space, potentially hindering generalization when the constraint is relaxed on larger maps.

c) Insufficient training budget for both agents.: The KL divergence and clipping fraction curves at the end of Essai 4 both indicate that the PPO policy had not yet converged within the 300k-step budget, which was chosen primarily for computational efficiency; training to 1M–5M steps, as is common in the VizDoom literature [2], would likely yield a more stable and capable policy. Similarly, although Arnold was trained for 3.27M steps and achieved stable kill statistics across all four phases, the phase-by-phase analysis in Table IX shows that mean kill count and reward variance did not improve meaningfully beyond Phase 1, suggesting that the agent reached a local behavioral plateau rather than a true optimum. Extending Arnold's training to 10M–20M steps, combined with a decaying learning rate schedule and periodic target-network resets, could allow the value function to escape this plateau and develop more sophisticated combat strategies on the larger maps.

d) Policy instability and catastrophic forgetting.: All four PPO experiments exhibited oscillatory reward curves with

no monotonic convergence, and Essai 3 showed a sharp performance degradation at 550k steps. These are hallmarks of PPO operating with too high a learning rate or entropy coefficient in a stochastic environment. Future work should explore through many experimentd the learning rate scheduling, KL-penalty PPO variants, or population-based training to improve stability.

e) Single-seed evaluation.: All training runs were performed with a single random seed, making it difficult to distinguish genuine learning trends from lucky or unlucky initializations. Multi-seed experiments with more than 5M timesteps and statistical significance tests are necessary to draw robust conclusions about the relative performance of the two approaches.

f) Limited action space.: The eight macro-actions available to the PPO agent do not cover strafing while turning or crouching, which are standard combat mechanics in Doom. Expanding the action set (possibly with action-space factorization as used in Arnold) could improve the agent's tactical flexibility and enable more nuanced combat behavior.

VII. Conclusion

This paper presented FreeDoom, a comparative study of two deep reinforcement learning agents for first-person combat in VizDoom deathmatch: a PPO-based agent trained in a spatially constrained arena, and Arnold, a Dueling DRQN agent trained on a full deathmatch map across five rotating maps.

The PPO investigation, structured as four iterative experiments, demonstrated that careful reward engineering is at least as important as algorithm choice in sparse adversarial environments. By correcting a misattributed damage signal, normalizing the kill reward, and introducing a bounded spatial arena, the final model (`doom_ppo_v1_3`) achieved a 50 % success rate on a 10-episode test after only 301k training timesteps. The consistency of successful episodes — reward ≈ 2.18, steps ≈ 167, indicating that a repeatable winning strategy has been partially internalized, even though PPO diagnostic metrics confirm the policy had not yet converged.

Arnold, building on the game-feature architecture of Lample and Chaplot [2], demonstrated that Dueling DRQN scales effectively to richer environments: 3.27 million training steps yielded a mean of 16.8 kills per episode and a K/D ratio of 10.68, with stable performance across all four training phases and no signs of catastrophic forgetting. The auxiliary game-feature head proved its value as a perceptual regularizer, co-converging with the TD loss without interference.

Together, the two approaches highlight a fundamental trade-off: PPO provides fast iteration cycles and interpretable training diagnostics, making it well-suited for rapid prototyping of reward functions in simplified settings; Dueling DRQN, while requiring a significantly larger training budget and a more complex architecture, scales to richer and more adversarial scenarios without environment simplification.

Future work will pursue several directions to address the identified limitations, adding longer training runs with multi-seed evaluation for the PPO agent, dynamic spatial curricula that gradually expand the accessible zone as skill improves, custom ACS instrumentation to expose outgoing damage as an intermediate reward signal, and a direct head-to-head comparison of the two approaches on a shared map and opponent configuration to isolate the contribution of architecture from that of environment design.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347 [cs.LG]*, Jul. 2017.

[2] G. Lample and D. S. Chaplot, "Playing FPS games with deep reinforcement learning," *arXiv preprint arXiv:1609.05521 [cs.AI]*, Sep. 2016, doi: 10.48550/arXiv.1609.05521.

[3] M. Kempka, M. Wydmuch, G. Runc, J. Toczek, and W. Jaśkowski, "ViZDoom: A Doom-based AI research platform for visual reinforcement learning," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, Santorini, Greece, Sep. 2016, pp. 1–8, doi: 10.1109/CIG.2016.7860433.

[4] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Savannah, GA, USA, Nov. 2016, pp. 265–283.

[5] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," in *Proc. 39th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025, arXiv:2407.17032.

[6] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, New York, NY, USA, Jun. 2016, pp. 1995–2003.

[7] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *Proc. AAAI Fall Symp. Sequential Decision Making Intelligent Agents*, 2015, arXiv:1507.06527.

[8] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.